

CSE201: Monsoon 2022  
Advanced Programming

# **Lecture 19: Introduction to Design Patterns**

Raghava Mutharaju

CSE, IIIT-Delhi

[raghava.mutharaju@iiitd.ac.in](mailto:raghava.mutharaju@iiitd.ac.in)

# What is Design Pattern

- It is a solution for a repeatable problem in the software design
- This is not a complete design for a software system that can be directly transformed into code
- It is a description or template for how to solve the problem that can be used in many different situations

# Why Study Patterns

## ● Reuse tried, proven solutions

- Provides a head start
- Avoids gotchas later (unanticipated things)
- No need to reinvent the wheel

## ● Establish common terminology

- Design patterns provide a common point of reference
- Easier to say, “We could use Strategy here.”

## ● Provide a higher level prerspective

- Frees us from dealing with the details too early

# “GoF” (Gang of Four) patterns

## ● Creational Patterns

*(abstracting the object-instantiation process)*

- Factory Method      Abstract Factory      Singleton
- Builder      Prototype

## ● Structural Patterns

*(how objects/classes can be combined)*

- Adapter      Bridge      Composite
- Decorator      Facade      Flyweight
- Proxy

## ● Behavioral Patterns

*(communication between objects)*

- Command      Interpreter      Iterator
- Mediator      Observer      State
- Strategy      Chain of Responsibility      Visitor
- Template Method

# Pattern: Iterator

*objects that traverse collections*

# Pattern: Iterator

## ●Recurring Problem

- How can you loop over all objects in any collection. You don't want to change client code when the collection changes. Want the same methods

## ●Solution

1. Provide a standard *iterator* object supplied by all data structures
2. The implementation performs traversals, does bookkeeping
3. The implementation has knowledge about the representation
4. Results are communicated to clients via a standard interface

## ●Consequences

- Can change collection class details without changing code to traverse the collection

# Java Iterator Recap

# Iterable Interface Source Code

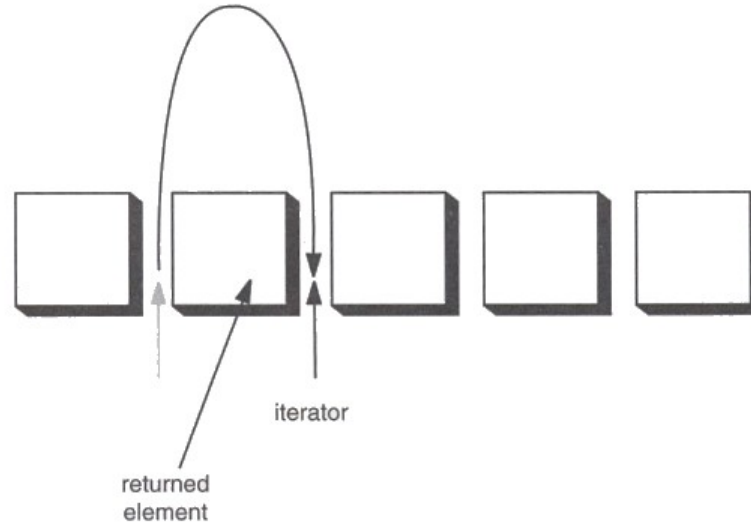
```
package java.lang;  
public interface Iterable<E> {  
    Iterator<E> iterator();  
}
```

- Just one method in this interface
- Objects of all classes that implements this interface can be the target of foreach statement
- Iterators allow iterating over the entire collection. It also allows element removal from collection during iteration

# Iterator Interface

- Defines three fundamental methods
  - `Object next()`
  - `boolean hasNext()`
  - `void remove()`
- These three methods provide access to the contents of the collection
- An Iterator knows position within collection
- Each call to `next()` “reads” an element from the collection
  - Then you can use it or remove it

# Iterator Position



**Figure 2-3: Advancing an iterator**

# Collection Interface Source Code

```
package java.util;
public interface Collection<E> extends
Iterable<E> {
    int size();
    boolean isEmpty();
    contains(Object o);
    boolean add(E e);
    boolean remove(Object o);
    equals(Object o);
    .....
    .....
}
```

- Defines fundamental methods that are enough to define the basic behavior of a collection
- Inherit the method from Iterable interface

# Example - SimpleCollection

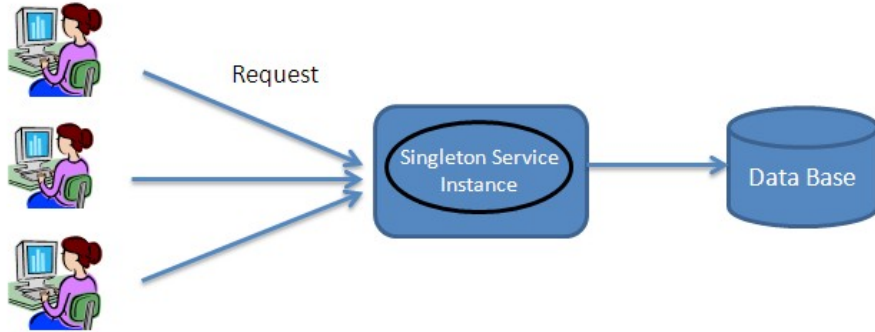
```
public class SimpleCollection {  
    public static void main(String[] args) {  
        Collection c = new ArrayList();  
        for (int i=0; i < 10; i++) {  
            c.add(i);  
        }  
        Iterator iter = c.iterator();  
        while (iter.hasNext())  
            System.out.println(iter.next());  
    }  
}
```

# Pattern: Singleton

*A class that has only a single instance*



# Pattern: Singleton



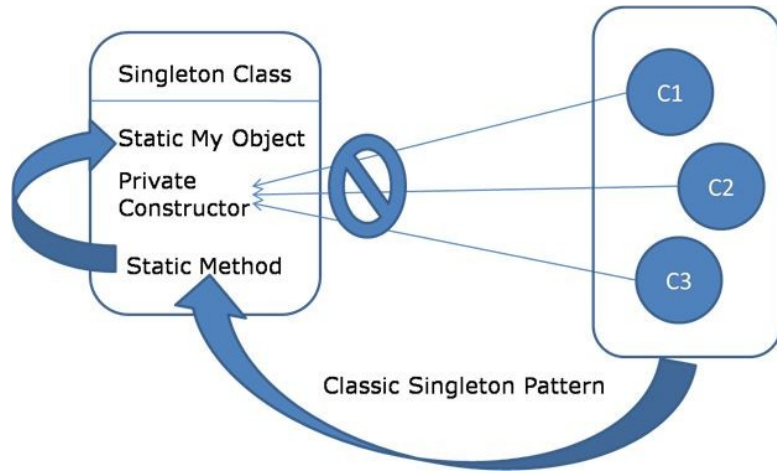
## ● Recurring problem

- Sometimes we only ever need one instance of a particular class
- It should be illegal to have another instance of the same class

## ■ Solution

- Singleton pattern – ensuring that a class has at most one instance
- Providing global access to that instance

# Implementing Singleton



1. Make constructor private so that no client is able to call it from outside
2. Declare a single private static instance of the class
3. Write a `getInstance()` method (or similar) that allows access to the single instance
  - o Ensure thread safety in case multiple threads can access this method

# Singleton Example

```
public class RandomGenerator {  
    private static RandomGenerator gen = null;  
    public static RandomGenerator getInstance()  
    {  
        if (gen == null) {  
            gen = new RandomGenerator();  
        }  
        return gen;  
    }  
    private RandomGenerator() {}  
    ...  
}
```

- Creates a new random generator
- Clients will not use the constructor directly but will instead call getInstance to obtain a RandomGenerator object that is shared by all classes in the application
- Lazy initialization
  - Can wait until client asks for the instance to create it
  - How to ensure thread safety?

# Singleton Comparator

```
public class LengthComparator
    implements Comparator<String> {
    private static LengthComparator comp = null;
    public static LengthComparator getInstance()
    {
        if (comp == null) {
            comp = new LengthComparator();
        }
        return comp;
    }
    private LengthComparator() {}
    public int compare(String s1, String s2) {
        return s1.length() - s2.length();
    }
}
```

- Comparators make great singletons because they have no state
- Saves memory by not allowing the creation of more than one object

# Pattern: Flyweight

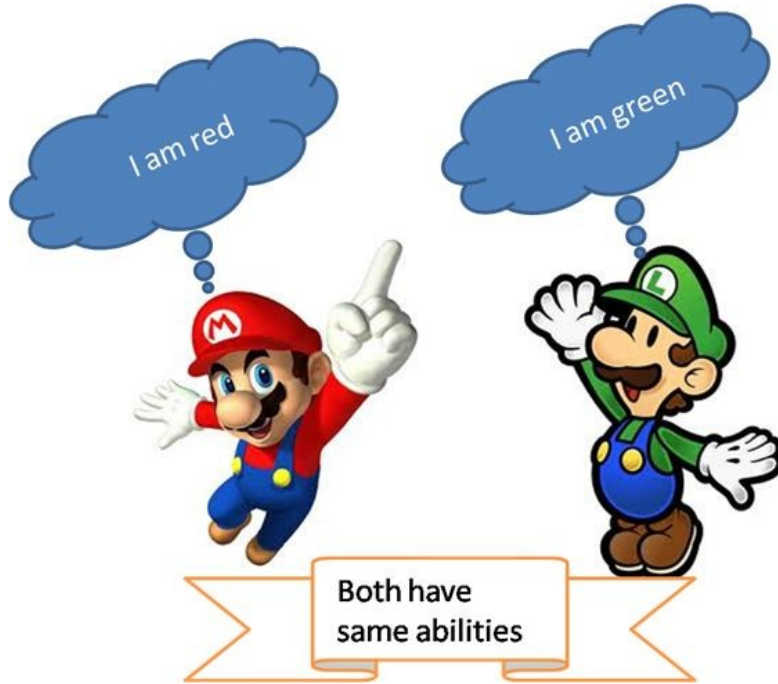
*a class that has only one instance for  
each unique state*

# Pattern: Flyweight

## ● Problem

- **Redundant objects** can bog down the system
  - Many objects have the same state
- Example: File objects that represent the same file on disk
  - `new File("chatlog.txt")`
  - `new File("chatlog.txt")`
  - `new File("chatlog.txt")`
  - ...
  - `new File("notes.txt")`
- Example: Date objects that represent the same date of the year
  - `new Date(4, 18)`
  - `new Date(4, 18)`

# Pattern: Flyweight



- An assurance that no more than one instance of a class will have identical state
  - Achieved by caching identical instances of objects.
  - Similar to singleton, but one instance for each unique object state
  - Useful when there are many instances, but many are equivalent

# Implementing a Flyweight (1/2)

```
public class Flyweighted {  
    private static Map<KeyType, Flyweighted> instances  
        = new HashMap<KeyType, Flyweighted>();  
  
    private Flyweighted(...) { ... }  
  
    public static Flyweighted getInstance(KeyType key) {  
        if (!instances.contains(key)) {  
            instances.put(key, new Flyweighted(key));  
        }  
        return instances.get(key);  
    }  
}
```

# Implementing a Flyweight (2/2)

```
public class Point {
    private int x, y;

    public Point(int x, int y) {
        this.x = x;
        this.y = y;
    }

    public int getX() { return x; }
    public int getY() { return y; }

    public String toString() {
        return "(" + x + ", " + y + ")";
    }
}
```



```
public class Point {

    private static Map<String, Point> instances =
        new HashMap<String, Point>();

    public static Point getInstance(int x, int y)
    {
        String key = x + ", " + y;
        if (!instances.containsKey(key)) {
            instances.put(key, new Point(x, y));
        }
        return instances.get(key);
    }

    private final int x, y; // immutable

    private Point(int x, int y) {
        this.x = x;
        this.y = y;
    }

    public int getX() { return x; }
    public int getY() { return y; }
    public String toString() {
        return "(" + x + ", " + y + ")";
    }
}
```

# Flyweighting in String by JVM

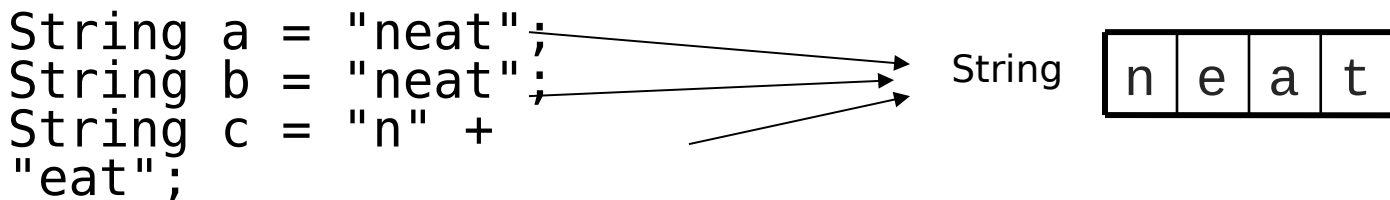
- The possible combinations for Strings is close to infinite, hence JVM maintains a cache for strings, called the **string constant pool**

- It is empty at startup and is filled constantly during the lifecycle of the JVM

- Java String objects are automatically flyweighted by the JVM **whenever** possible

- If you declare two string variables that point to the same literal.

- If you concatenate two string literals to match another literal



# Next Lecture

- More design patterns